

Beginner's Guide To Java Unit Tests - Spike Work

Group 6

March 11, 2025

Contents

1	Introduction	2
1.1	General J unit tests	2
1.2	Outline of a Test class and Test methods	2
2	How to make Tests using Inteli-J	4
2.1	Step 1	4
2.2	Step 2	4
2.3	Step 3	5
2.4	Step 4	5
2.5	Step 5	6
2.6	Step 6	6
3	Java FX - J Unit tests	7
3.1	Best practices	7

Chapter 1

Introduction

NIALL PERRY

JUnit is a widely used framework for unit testing in Java. In this guide, I will show how to perform general Java unit tests and then Java unit tests for JavaFx.

1.1 General J unit tests

What is Unit Testing? A procedure to validate individual units of the source code

Example: A procedure, method, or class

Validating each individual piece reduces errors when integrating the pieces together later.

A recommendation is to have tests that focus on boundaries of allowed data and where erroneous data are handled correctly...

If a method takes data 0.. 10 What should you test? Answer: -1, 0, 10, 11

1.2 Outline of a Test class and Test methods

Test class

```
1
2      package group.tests;
3      import org.junit.jupiter.api.Assertions;
4      import org.junit.jupiter.api.BeforeEach;
5      import org.junit.jupiter.api.Test;
6      import group6.*;
7
8      public class AlbumTest {
9          private Album album;
10
11         @BeforeEach
12         public void setup(){
13             album = new Album();
14         }
15
16         @Test
17         public void testAddOneSongAndFind() {
18             // Test code here
19         }
20     }
```

Tests method marked with @Test is run...

@BeforeEach methods are always run before each @Test method...

There are also @AfterEach, @BeforeAll, @AfterAll

Test methods

```
1      @Test
2      public void testAddOneSongAndFind() { // [1]
3
4          // Set up test fixture
5          Contact contact = new Contact("Hey Jude", "The Beatles"); // [2]
6
7          // Exercise the program under test [3]
8          album.add(contact);
9          Song foundSong = album.search("Hey Jude");
10
11         // Check expected data against data returned [4]
12         Assertions.assertEquals(song, foundSong,
13             "Incorrect song found!");
14     }
```

Things to note:

[1] Meaningful name

[2] Create the test data: called the test fixture

[3] Do something interesting and get a result

[4] Did we get the right result?

Mantra: Arrange, Act, Assert

Chapter 2

How to make Tests using Inteli-J

We will create a JUnit test to check that songs can be added to an album.

2.1 Step 1

Create package

1. Right-click on src
2. Select New
3. Select Package

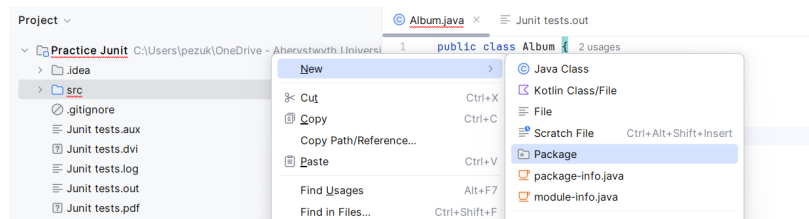


Figure 2.1: a visual guide

2.2 Step 2

Create test class

1. Right click on the tests package
2. Select New
3. Select Class

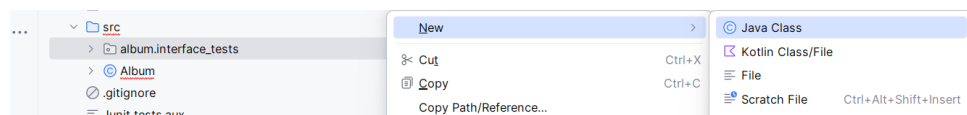


Figure 2.2: visual guide

2.3 Step 3

Adding to test class

1. Write some code
2. Import the JUnit library

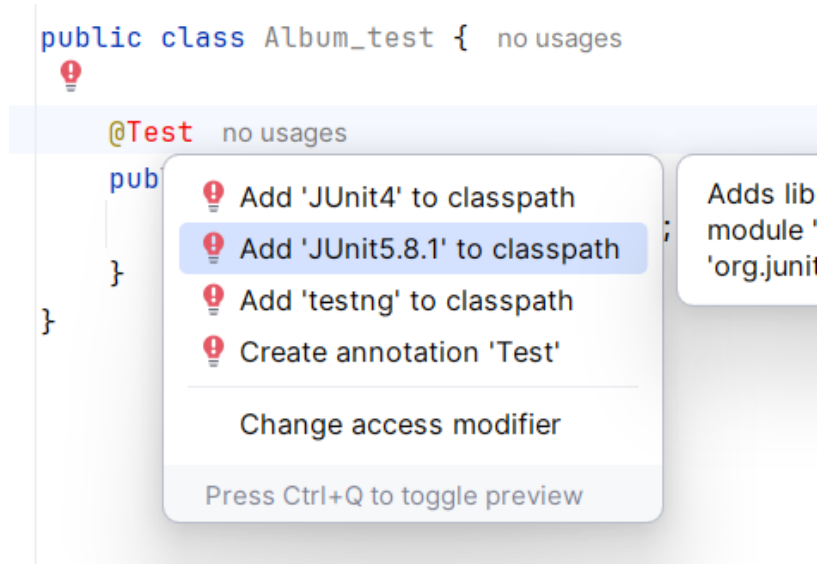


Figure 2.3: visual guide

2.4 Step 4

Run the test class

1. run code



Figure 2.4: visual guide

Here is the expected output...

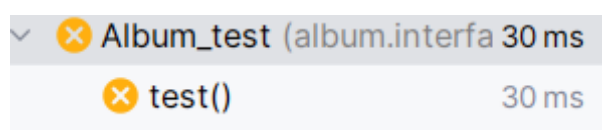


Figure 2.5: Result of action

2.5 Step 5

Add proper test code

For example:

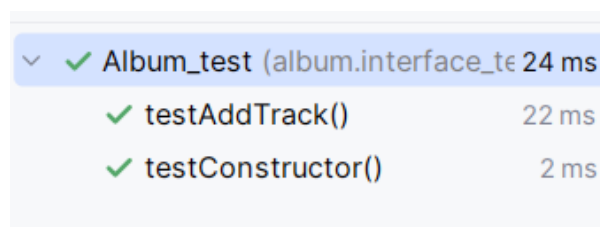
```
1      public class Album_test {
2
3          public Album album;
4          // initialises test
5          @BeforeEach
6          void setUp() {
7              album = new Album("Michael Jackson", 1982, "Thriller");}
8
9          // example of testing constructor
10         @Test
11         void testConstructor() {
12             assertEquals("Thriller", album.getAlbumTitle());
13             assertEquals("Michael Jackson", album.getArtist());
14             assertEquals(1982, album.getYear());
15             assertTrue(album.getTracks().isEmpty(), "Track list should
              be empty initially");}
16
17         // example of testing track addition
18         @Test
19         void testAddTrack() {
20             album.addTracks("Billie Jean");
21             album.addTracks("Beat It");
22             List<String> tracks = album.getTracks();
23             assertEquals(2, tracks.size());
24             assertTrue(tracks.contains("Billie Jean"));
25             assertTrue(tracks.contains("Beat It"));
26         }
27     }
```

*** This can only be done if the method tested exists.

2.6 Step 6

finally

1. compile the code



✓ Album_test (album.interface_te 24 ms)	
✓ testAddTrack()	22 ms
✓ testConstructor()	2 ms

Figure 2.6: the result

Chapter 3

Java FX - J Unit tests

3.1 Best practices

1. Use TestFX for UI Interactions

```
1 import org.testfx.api.FxRobot;
2
3 class MainAppTest extends ApplicationTest {
4
5     private Button button;
6
7     @Override
8     public void start(Stage stage) {
9         button = new Button("Click Me");
10        button.setOnAction(event -> button.setText("Clicked!"));
11
12        stage.setScene(new javafx.scene.Scene(new javafx.scene.layout.
13            StackPane(button), 300, 200));
14        stage.show();
15    }
16
17    @Test
18    void testButtonClick(FxRobot robot) {
19        assertEquals("Click Me", button.getText()); // Verify initial
20            state
21
22        robot.clickOn(button); // Simulate a click
23
24        assertEquals("Clicked!", button.getText()); // Verify updated
25            text
26    }
27 }
```

JavaFX involves UI components that require a JavaFX thread to function correctly. TestFX provides utilities to interact with JavaFX elements.

For example, using FxRobot

2. Always Run JavaFX Tests on the JavaFX Thread

```
1 import javafx.application.Platform;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class JavaFXThreadTest {
6
7     @Test
8     void testFXThreadExecution() {
9         Platform.runLater(() -> {
10             System.out.println("Running on JavaFX Thread!");
11             assertTrue(Platform.isFxApplicationThread());
12         });
13     }
14 }
```

3. Use @Start for Stage Initialization

Instead of setting up the UI in @BeforeEach, use @Start in TestFX.

@Start automatically launches a JavaFX stage, avoiding manual setup.

The start(Stage stage) method ensures UI elements are created properly.

```
1 public void start(Stage stage) {
2     button = new Button("Click Me");
3     button.setOnAction(event -> button.setText("Clicked!"));
4
5     stage.setScene(new javafx.scene.Scene(new javafx.scene.layout.
6         StackPane(button), 300, 200));
7     stage.show();
8 }
```

4. Use Timeouts for Asynchronous UI Operations

Some JavaFX operations run asynchronously (e.g., animations, background tasks). To avoid flaky tests, use TestFX's wait conditions.

```
1 import org.testfx.util.WaitForAsyncUtils;
2 import java.util.concurrent.TimeUnit;
3
4 WaitForAsyncUtils.sleep(1, TimeUnit.SECONDS); // Wait before checking
5 assertEquals("Clicked!", button.getText());
```

5. Use lookup() for Selecting UI Elements

```
1 @Test
2 void testButtonClick(FxRobot robot) {
3     robot.clickOn(".button"); // Find button by CSS class
4     assertThat(lookup(".button").queryButton()).hasText("Clicked!");
5 }
```

6. Use Assertions for UI Elements

Use TestFX Assertions to validate UI state.

```
1 import static org.testfx.assertions.api.Assertions.assertThat;
2
3 assertThat(button).hasText("Clicked!");
4 assertThat(label).isVisible();
5 assertThat(textField).hasText("Hello World!");
```